

Relatório Técnico para o Trabalho Final de Compiladores

Augusto Kist Lunardi
Leandro Oliveira Galbarino Do Nascimento

1 Introdução

Este relatório busca descrever o trabalho final da disciplina de compiladores, contemplando a implementação de um compilador para a linguagem Homi, projetada para abstrair a criação de automações para o sistema Home Assistant.

2 Ferramentas e Instalação

A implementação foi desenvolvida em Python 3, utilizando a biblioteca PLY para a geração dos autômatos léxicos e sintáticos. Para executar o projeto, é necessário instalar a dependência do PLY:

```
pip install ply
```

A execução do compilador pode ser feita através do Makefile disponibilizado, ou diretamente via interpretador Python: `python main.py exemplos/meu_script.homi`.

3 Gramática Livre de Contexto

Conforme solicitado, a linguagem foi estabelecida partindo da premissa de que seria compreensível por usuários não especializados, focando em gatilhos de tempo e estado, condições lógicas e chamadas de serviços com ou sem parâmetros. A gramática $G = (V, \Sigma, P, S)$ é definida formalmente a seguir.

3.1 Conjunto de Variáveis Não-Terminais (V)

$$V = \{\langle \text{programa} \rangle, \langle \text{automacao} \rangle, \langle \text{quando} \rangle, \langle \text{gatilho} \rangle, \langle \text{se} \rangle, \langle \text{condicoes} \rangle, \langle \text{condicao} \rangle, \langle \text{entao} \rangle, \langle \text{acoes} \rangle, \langle \text{acao} \rangle, \langle \text{parametro} \rangle, \langle \text{empty} \rangle\}$$

3.2 Conjunto de Símbolos Terminais (Σ)

$$\Sigma = \{\text{AUTOMACAO}, \text{QUANDO}, \text{SE}, \text{ENTAO}, \text{FIM}, \text{STRING}, \text{ENTITY_ID}, \text{TEMPO}, \text{ESTADO}, \text{NUMERO}, \text{OP_RELACIONAL}, \text{ATRIBUICAO}, \text{PALAVRA_TEMPO}, \text{SERVICO}, \text{PONTO}, \text{ABRE_PAR}, \text{FECHA_PAR}, \text{PONTO_VIRGULA}, \text{E}, \text{OU}\}$$

3.3 Regras de Produção (P)

$\langle \text{programa} \rangle \rightarrow \langle \text{automacao} \rangle$
 $\langle \text{automacao} \rangle \rightarrow \text{AUTOMACAO STRING } \langle \text{quando} \rangle \langle \text{se} \rangle \langle \text{entao} \rangle \text{ FIM}$
 $\langle \text{automacao} \rangle \rightarrow \text{AUTOMACAO STRING } \langle \text{quando} \rangle \langle \text{entao} \rangle \text{ FIM}$
 $\langle \text{quando} \rangle \rightarrow \text{QUANDO } \langle \text{gatilho} \rangle$
 $\langle \text{gatilho} \rangle \rightarrow \text{ENTITY_ID OP_RELACIONAL ESTADO}$
 $\langle \text{gatilho} \rangle \rightarrow \text{PALAVRA_TEMPO ATRIBUICAO TEMPO}$
 $\langle \text{se} \rangle \rightarrow \text{SE } \langle \text{condicoes} \rangle$
 $\langle \text{condicoes} \rangle \rightarrow \langle \text{condicao} \rangle \text{ E } \langle \text{condicoes} \rangle \mid \langle \text{condicao} \rangle \text{ OU } \langle \text{condicoes} \rangle \mid \langle \text{condicao} \rangle$
 $\langle \text{condicao} \rangle \rightarrow \text{ENTITY_ID OP_RELACIONAL NUMERO}$
 $\langle \text{entao} \rangle \rightarrow \text{ENTAO } \langle \text{acoes} \rangle$
 $\langle \text{acoes} \rangle \rightarrow \langle \text{acao} \rangle \langle \text{acoes} \rangle \mid \langle \text{acao} \rangle$
 $\langle \text{acao} \rangle \rightarrow \text{ENTITY_ID PONTO SERVICO ABRE_PAR } \langle \text{parametro} \rangle \text{ FECHA_PAR PONTO_VIRGULA}$
 $\langle \text{parametro} \rangle \rightarrow \text{NUMERO} \mid \text{TEMPO} \mid \langle \text{empty} \rangle$
 $\langle \text{empty} \rangle \rightarrow \epsilon$

4 Analisador Léxico

A Análise Léxica foi implementada utilizando a estrutura de expressões regulares da biblioteca PLY, que gera um Autômato Finito Determinístico otimizado internamente.

O analisador atua como a porta de entrada do compilador, etiquetando sequências de caracteres e descartando ruídos como espaços em branco e comentários iniciados por #.

Os tokens simples foram declarados diretamente como variáveis, enquanto tokens complexos e palavras reservadas foram declarados através de funções, o que permite o retorno exato e exclusivo daquela estrutura.

O analisador também mantém uma contagem de quebras de linha ($\backslash n$) para possibilitar o reporte exato na ocorrência de caracteres que acarretam problemas ou tokens não reconhecidos.

Além disso, a ordem de declaração das funções considera que o PLY avalia as ERs sequencialmente. Sendo assim, tokens mais específicos foram colocados no topo do arquivo para evitar que fossem incorretamente absorvidos pelas ERs de tokens mais genéricos.

Analisador Sintático O Analisador Sintático processa o fluxo de tokens gerado pelo Lexer e constrói a Árvore Sintática Abstrata (AST) validando a estrutura contra a GLC estabelecida. A linguagem utiliza um Parser Bottom-Up da família LR(k), construído com o algoritmo LALR(1) através do módulo yacc do PLY. A Tabela Preditiva LR não foi escrita à mão no código Python; o PLY compila a gramática e gera a Tabela de Transição de Estados automaticamente em background.

No PLY, cada função define as regras de produção da nossa gramática. O símbolo não-terminal do lado esquerdo recebe os dados processados dos símbolos do lado direito. O parser agrupa essas informações montando um dicionário em Python, onde a chave 'node' funciona apenas como uma etiqueta para identificar qual é a estrutura, e as outras chaves guardam os dicionários dos nós filhos, construindo assim a nossa AST de baixo para cima, facilitando o posterior percurso da árvore.

Nesse cenário, foi implementada a técnica de recuperação de erros baseada no Modo Pânico: caso o parser encontre um token inesperado, ele descarta os tokens subsequentes até localizar um token de sincronização pré-definido ou a palavra-chave **Fim**, permitindo que a análise continue e múltiplos erros possam ser detectados em uma única execução.

5 Analisador Semântico

A Análise Semântica garante a consistência lógica e a conformidade dos tipos operacionais antes da geração do código final, percorrendo a AST de forma estrutural utilizando o padrão Visitor. A abordagem utilizada baseia-se na construção de uma Tabela de Símbolos.

Esta tabela registra os domínios mapeados das entidades e estabelece regras rígidas de acesso e tipos paramétricos:

- **Consistência Externa:** Impede que domínios incompatíveis executem ações que não lhes pertencem.
- **Verificação de Tipagem Estrita de Parâmetros:** Verifica os argumentos passados para os serviços.

6 Geração de Código Intermediário (YAML)

A etapa final do compilador converte a AST validada pelo analisador semântico no arquivo YAML exigido pelo Home Assistant. A principal abordagem técnica desta fase é o controle manual de indentação de strings e a estruturação dos blocos condicionais. Os operadores relacionais da linguagem fonte são mapeados diretamente para as chaves nativas do sistema destino.

7 Exemplos de Scripts Homi e YAMLs Resultantes

Abaixo são apresentados dois exemplos contemplando os recursos desenvolvidos.

7.1 Exemplo 1

Código Fonte (Homi):

```
Automacao "Desligar luzes      meia-noite"
Quando tempo = 00:00
Entao light.casa_toda.turn_off();
Fim
```

Saída YAML Gerada:

```
- alias: 'Desligar luzes      meia-noite'
  description: 'Gerado pelo Compilador Homi'
  mode: single
  triggers:
    - trigger: time
      at: '00:00'
  conditions: []
```

```
actions:
- action: light.turn_off
  target:
    entity_id: light.casa_toda
```

7.2 Exemplo 2

Código Fonte (Homi):

```
Automacao "Ligar ar condicionado no fim da tarde"
Quando tempo = 18:00
Se sensor.temperatura_quarto > 25
Entao switch.ar_condicionado.turn_on();
Fim
```

Saída YAML Gerada:

```
- alias: 'Ligar ar condicionado no fim da tarde'
  description: 'Gerado pelo Compilador Homi'
  mode: single
  triggers:
  - trigger: time
    at: '18:00'
  conditions:
  - condition: numeric_state
    entity_id: sensor.temperatura_quarto
    above: 25
  actions:
  - action: switch.turn_on
    target:
      entity_id: switch.ar_condicionado
```